



Predicting Pedestrian Flow

A 60-minute forecasting system for urban footfall

A live machine-learning system that forecasts pedestrian traffic at a monitored urban location one hour into the future, using public mobility, weather and temporal context.

WHO	Businesses operating in high-footfall areas: shops, restaurants and other local operators that benefit from anticipating near-term demand. The system is designed around public data, with private customer-count integration left as a future extension.
WHERE	Zürich, Switzerland. The initial target is the Hystreet counter at Lintheschergasse, with public-transport activity around Zürich HB as future context.
WHEN	Primary prediction horizon: the next 60-minute interval. The inference system runs on a schedule; the same forecasting step can be run recursively to produce longer-range forecasts, with uncertainty compounding across steps.
WHAT	Predict the number of pedestrians recorded during the next hour. The target is pedestrian flow, not an estimate of shoppers, commuters or people physically present at an instant.
WHY	Near-term footfall is an operational signal. A reliable forecast can support staffing, preparation, stock decisions and other time-sensitive planning while remaining based on data a small business could plausibly access.

Core question

Can upcoming public-transport activity, together with recent pedestrian flow, weather and calendar context, improve a 60-minute pedestrian-flow forecast over a strong seasonal base-line?

Author
Timo Schlumpf

Repository
github.com/0xff4b/gaeja

1 Problem statement

Businesses in high-footfall areas can benefit from knowing how busy their surroundings will be before the next hour begins. This project predicts the **total number of pedestrians recorded at a monitored location during the next 60-minute interval**, using only information available at prediction time.

The initial target will be the Hystreet measurement area at Lintheschergasse in Zürich, with the other Hystreet areas used as spatial context. The City of Zürich publishes measurements supplied by hystreet.com GmbH; they are updated hourly and provide more than five years of historical coverage, with the dataset continuing to grow by new hourly observations. The primary success criterion is a **relative 10% reduction in MAE versus a seasonal-naive baseline**: MAE must be at most 90% of the baseline MAE. The seasonal-naive forecast is the observed count from the same location one week earlier (t-168 hours). The 10% margin is intended as a meaningful but deliberately modest improvement threshold: it requires a measurable gain over a strong temporal baseline without imposing an arbitrary absolute error target. Persistence will be reported as a second baseline. Evaluation will use an out-of-time test period.

2 Originality & motivation

The intended product is a forecasting service for shops, restaurants and other businesses where pedestrian traffic affects staffing, preparation, stock and operations. It deliberately forecasts an observable quantity: counted pedestrians are not assumed to be customers. A future store integration could calibrate public footfall against private customer-count data. The project also builds on an existing weather-data ingestion prototype, making the initial data-engineering scope realistic while leaving the ML and MLOps layers as the core work.

The central modelling idea is to treat **upcoming public-transport services as future-known structured context**. Instead of using only a scalar “trains next hour” feature, the model receives the scheduled events in the prediction horizon, including their timing and service characteristics. The project therefore asks whether the composition of future mobility improves short-horizon pedestrian forecasting beyond historical flow, weather and calendar context. This differentiates the project from generic traffic forecasting. The concept was checked against previous HSLU and KTH ID2223 project topics.

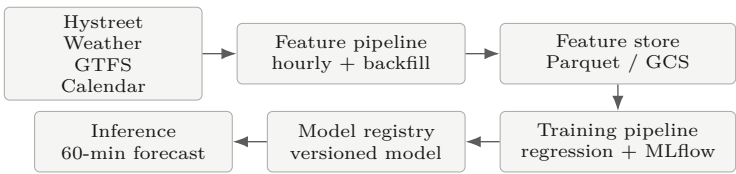
3 Data source & features

Source	Used for	Notes
Hystreet / Zürich Open Data	Target; lags 1, 2, 3, 6, 24, 168 h; rolling means 3, 6, 24, 168 h; spatial context	Public CSV/Parquet dataset supplied by hystreet.com GmbH; updated hourly; no scraping; historical + ongoing.
Zürich weather	Temperature, precipitation, humidity, pressure, wind	REST API + historical downloads; only values available at prediction time are used.
Swiss GTFS	Upcoming public-transport events	Dated timetable snapshots; each event encoded by time-to-arrival/departure, service type and line.
Zürich school holidays	Hour, weekday, weekend, public holiday, school holiday	Deterministic calendar features.

The label is the observed pedestrian count in the future hourly interval and is not mechanically derived from the inputs. This is a continuous regression target; rare-positive-class handling is not applicable. Backfill uses only information available at each historical prediction timestamp. For GTFS, the latest archived timetable snapshot available on or before the timestamp is used when its validity covers the prediction horizon; later revisions are excluded. Future pedestrian observations, future weather and final train outcomes are never inputs. Missing sensor intervals are handled from timestamps rather than row position. GTFS-RT delays and train-occupancy forecasts remain optional extensions.

4 System design

The project follows the required Feature–Training–Inference (FTI) architecture. The core system prioritizes reliable ingestion, backfill, feature storage, reproducible training and serving.



The feature pipeline runs on a schedule and supports historical backfill. The training pipeline retrains on matured labels, evaluates against the baselines, tracks experiments with MLflow and registers the selected model. The inference pipeline joins the current feature state with retrieved upcoming transport events and serves the next-hour forecast through a lightweight web UI. Once the interval has elapsed, the observed footfall count becomes a matured label for a later training run.

Python	ingestion, feature computation and model code
Docker	reproducible pipelines and inference
GitHub Actions	scheduled pipelines and CI
Parquet / GCS	versioned feature storage and backfill
MLflow	experiment tracking and model registry
FastAPI + web dashboard	prediction serving and live model output

Feature definitions will be shared between training and inference. The repository is public and kept reachable throughout the semester. Optional extensions include GTFS-RT disruptions, train-occupancy context, richer event data and additional monitoring; the core FTI system is built first.